CS 470: Assignment 02

Programming Assignments (90%)

Your program MUST be written in **Python**. Your code file should be named **A02.py.**

The goals of this assignment are to perform convolution using various approaches. You will also create a Gradio interface of your own design to demonstrate the functionality of your code.

Restrictions

You CANNOT use the following functions:

- **cv2.filter2D (or any other convolution/filtering functions in OpenCV)**
- **np.convolve**
- **scipy.signal.convolve2d**

The following are permitted in all functions:

- **cv2.copyMakeBorder()**
- **np.pad()**
- **cv2.flip()**
- **cv2.convertScaleAbs()**

If you are not sure whether a library function is permitted, ASK ME!

EXCEPT for do_convolution_slow(), while and for loops must be avoided. Everything can be implemented by taking advantage of numpy's indexing and broadcasting functionality.

A02.py

The following general guidelines apply:

- All input images are grayscale images of shape (height, width) and with dtype "uint8".
- Input images and kernels should be converted to **"float64" before calculations**.
- All padding (actual or implicit) is assumed to be **zero-padding**.
- Output images are expected to have the same shape, but type will either be "uint8" or "float64". If convert_uint8 is True, ***use cv2.convertScaleAbs() with the provided alpha and beta to convert (as opposed to the numpy variant):***
    - **output = cv2.convertScaleAbs(output, alpha=alpha, beta=beta)**
    - **WARNING: Use the keyword arguments for alpha and beta as shown above!**
- All convolution functions take the same parameters and should return the same output given the same parameter values (they only differ in approach).

You are welcome and in fact encouraged to write helper functions for this assignment, BUT the below functions should be written as specified:

- **def read_kernel_file(filepath)**
  - Open a file for reading and grab the first line
  - The kernel files have the format:
    - (row count) (column count) (value 0,0) (value 0,1) … (value r,c)
  - For example:
    - 3 3 1 2 1 0 0 0 -1 -2 -1
  - …will ultimately become:
    - 1 2 1
      0 0 0
      -1 -2 -1
  - Split the line into tokens by spaces
    - Use the string split method
  - Grab the first and second tokens, convert them to ints, and store them as the row and column counts:
    - rowCnt = int(tokens[0])
    - colCnt = int(tokens[1])
  - Starting at index = 2, convert the remaining tokens into a kernel array with type "float64" and shape (rowCnt, colCnt)
    - ***HINT***: You CAN do this without a loop, as numpy CAN store string types (dtype=np.str_). You can then convert to float.
  - Return your kernel

- **def do_convolution_slow(image, kernel, alpha=1.0, beta=0.0, convert_uint8=True)**
  - Implement convolution ***using pure Python code and loops***.
  - **Avoid library functionality (OpenCV, Numpy, etc.) EXCEPT for:**
    - Numpy arrays (creation, indexing, slicing, shape)
    - The "always permitted" functions above
  - ***This is the ONLY function in this assignment where loops are not only allowed but required.***

- **def do_convolution_fast(image, kernel, alpha=1.0, beta=0.0, convert_uint8=True)**
  - Implement convolution using numpy (this time avoiding loops).

- **def do_convolution_fourier(image, kernel, alpha=1.0, beta=0.0, convert_uint8=True)**
  - Implement convolution using the Fourier transform.
  - Make sure to use **cv2.getOptimalDFTSize()** for efficiency.

- **def main()**
    - When running A02, your program should start a Gradio interface (that can be accessed through a browser; see the terminal for the address).
    - You are largely free to design this Gradio interface as you see fit with the following requirements:
        - Allow the user to load an image (with image_mode = "L").
        - Allow the user to load a kernel file (using gr.File()).
        - Allow the user to select alpha and beta numbers for conversion.
        - Assume that convert_uint8 is always True.
        - Allow the user to filter the image (using either do_convolution_fast() or do_convolution_fourier()) with the loaded kernel file.
        - You must display the filtered image (with image_mode = "L").
        - The interface should work correctly and as expected.
        - The interface should be reasonably organized.

Testing Screenshot (5%)

I have provided several new files for testing:

- **Test_A02.py** – the test program
- **assign02/**
    - **images/**
        - Input images for testing
    - **filters/**
        - Filter files

Once you have merged from my repo into yours, these files/folders should reside in your main project directory. Your source file (A02.py) should reside in the main project directory as well (NOT the assign02 folder). You can either run the testing programs directly OR you can use the testing section of Visual Code.

You MUST run the tests and send a screenshot of the test results! Even if your program(s) do not pass all the tests, you MUST send this screenshot!

You may have to do "Command Palette" → "Python: Configure Tests" → unitttest → root directory → "test_*.py", and then modify .vscode/settings.json to use "Test_*.py"

This screenshot should show clearly:

- The final result of the test run on the command line ("OK" for all passing, "FAILED (failures=N)" for some or all failing).
- OR
- The testing view in Visual Code

The test programs also do NOT test the Gradio functionality.

The screenshot should be copied to the screenshots/ folder.

README.md (5%)

Update the README.md of your repository with the following:

- Add a new entry under "Runnable Python Scripts" for "A02.py". Very briefly describe what functionality is available and what happens if you run the program. I am only looking for a few sentences.

Grading

Your OVERALL assignment grade is weighted as follows:

- 5% - Testing results screenshot
- 5% - README update
- 90% - Programming assignments

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	**Maximum Penalty**
Code that is not syntactically correct	60
Usage of forbidden function(s)	60
Incorrect implementation of filter functions	60
Usage of for/while loops **per function** (or lack of it in do_convolution_slow())	10
Incorrect implementation of read_kernel_file()	10
Failing to either flip kernel (actual or conceptual)	5
Incorrect conversion to uint8	5
Incorrect padding for optimal DFT size	5
Bad Gradio interface	5
Technically correct code, but poor code design	5
Missing assignment Git branch	5
Missing testing screenshot	5
README update missing/incorrect	5

In general, these are things that will be penalized:

- **Code that is not syntactically correct (up to 60 points off!)**
- Sloppy or poor coding style
- Bad coding design principles
- Code that crashes, does not run, or takes a VERY long time to complete

- Using code from ANY source other than the course materials
- Collaboration on code of ANY kind; this is an INDIVIDUAL PROJECT
- Sharing code with other people in this class or using code from this or any other related class
- Output that is incorrect
- Algorithms/implementations that are incorrect
- Submitting improper files
- Failing to submit ALL required files